

Guía de Pruebas en Postman — Proyecto Numerología

Esta guía usa **los endpoints reales de su proyecto** (los del enunciado: Usuarios, MatricesNumerologicas, PromptsConfig, LecturasIA, BitacoraTransacciones), en lugar del ejemplo de la plataforma de música. El objetivo es que verifiquen, endpoint por endpoint, que el flujo completo funciona: crear usuario → calcular matriz → generar lectura con Gemini → consultar historial.

1. ¿Qué es Postman?

Postman es un **cliente HTTP** (aplicación de escritorio o extensión de navegador) que sirve para **enviar peticiones a una API sin necesidad de tener un frontend construido**. Ustedes ya tienen (o están construyendo) endpoints con Express — Postman es lo que les permite "hablarle" a esos endpoints directamente: eligen el método (POST, GET, etc.), la URL, el body en JSON, dan click en **Send**, y ven exactamente qué responde el servidor.

Con Postman pueden:

- Enviar peticiones a `http://localhost:3000` (o el puerto que definan) especificando método, headers y body
- Ver el código de estado (200, 201, 404, 500...), el JSON de respuesta y el tiempo que tardó
- Organizar las peticiones en **Collections** (carpetas)
- Guardar valores dinámicos (como el `_id` de un usuario recién creado) en **variables**, para reutilizarlos en la siguiente petición sin copiar y pegar a mano
- Automatizar la captura de esos valores con scripts en la pestaña **Tests**

Se descarga gratis desde postman.com/downloads. Alternativa liviana sin salir de VS Code: extensión "Thunder Client".

2. Preparación antes de abrir Postman

1. Verifiquen su `.env`, debe tener mínimo:

```
PORT=3000
MONGO_URI=mongodb://localhost:27017/numerologia_db
GEMINI_API_KEY=su_api_key_aqui
```

2. Levanten el servidor:

```
npm run dev
```

3. Confirмен en consola: MongoDB conectado correctamente y Servidor corriendo en `http://localhost:3000`. Si no aparece alguno de los dos, no continúen — revisen el `.env` primero.
-

3. Crear la Collection y el Environment en Postman

3.1 Collection:

1. **Collections** → + **Create Collection** → nómbrénla `Numerologia API`
2. Dentro, creen 3 carpetas (botón derecho → **Add folder**): `Usuarios`, `Numerologia`, `Lecturas IA`

3.2 Environment:

1. **Environments** → + → nómbrénlo `Local`
2. Variables:

Variable	Valor inicial
<code>base_url</code>	<code>http://localhost:3000/api/v1</code>
<code>usuario_id</code>	<i>(vacío, se llena al crear el usuario)</i>
<code>prompt_id</code>	<i>(vacío, se llena al crear un prompt activo)</i>

1. Seleccionen `Local` en el desplegable de arriba a la derecha antes de enviar cualquier petición — si no, `{{base_url}}` no se va a reemplazar.

4. Módulo de Gestión de Usuarios

4.1 Crear usuario — POST `{{base_url}}/usuarios`

Body → **raw** → **JSON**:

```
{
  "nombres": "Laura Valentina",
  "apellidos": "Ramírez Ortiz",
  "fecha_nacimiento": "1998-11-23",
  "email": "laura.ramirez@example.com"
}
```

Qué verificar:

- Código **201 Created**
- La respuesta trae `_id`, `fecha_creacion` (generado por el servidor, no lo enviaron ustedes)
- Prueba de validación: repitan la misma petición con el **mismo email** → debe fallar porque el schema lo define como único (`email: String, único`). Esperen un **400** o **409** según cómo lo hayan manejado en el controller — si les da **500**, es señal de que no están capturando el error de índice duplicado de MongoDB en el `catch`.

Guardar el `_id` automáticamente — pestaña **Tests** de esta petición:

```
const respuesta = pm.response.json();
pm.environment.set("usuario_id", respuesta._id);
```

4.2 Consultar usuario — GET

`{{base_url}}/usuarios/{{usuario_id}}`

- Esperado: **200 OK** con nombres, apellidos, fecha_nacimiento, email
- Prueben también con un id inventado
(`{{base_url}}/usuarios/000000000000000000000000`) → debe dar **404**

El enunciado no pide explícitamente PUT ni DELETE para Usuarios, pero si ustedes ya los implementaron (buena práctica de CRUD completo), agréguenlos aquí mismo con la misma lógica de las otras guías: PUT con datos válidos y con un dato que rompa una validación, DELETE seguido de un GET que confirme el 404.

5. Módulo de Cálculos Matemáticos (MatricesNumerologicas)

5.1 Ejecutar el cálculo — POST

`{{base_url}}/numerologia/calcular/{{usuario_id}}`

- No lleva body: el endpoint debe tomar nombres y fecha_nacimiento **del usuario ya guardado** (por eso el usuario_id va en la URL, no en el body)
- Esperado: **201 Created** (se creó un documento nuevo en MatricesNumerologicas) con numero_camino_vida, numero_expresion, numero_alma, todos reducidos a un dígito o a número maestro (11, 22, 33)
- **Prueba clave:** verifiquen a mano un caso simple (por ejemplo, sumar los dígitos de la fecha de nacimiento) y comparen contra lo que devuelve la API — si no coincide, el error está en el algoritmo de reducción numerológica, no en la ruta ni el modelo
- Prueben también llamando este endpoint con un usuario_id que no existe → debe dar **404**, no un 500 por intentar leer nombres de un usuario null

5.2 Consultar la matriz — GET

`{{base_url}}/numerologia/matriz/{{usuario_id}}`

- Esperado: **200 OK** con los 3 números calculados en el paso anterior
 - Si en el controller usan `.populate("usuario_id")`, la respuesta debe traer los datos del usuario embebidos en vez de solo el ObjectId — verifiquenlo explícitamente comparando la respuesta con y sin el `.populate()`
-

6. Módulo de Integración con IA (PromptsConfig + LecturasIA)

Antes de poder generar una lectura, tiene que existir **al menos un PromptsConfig con activo: true**, porque el endpoint de generación lo va a buscar en la base de datos.

6.1 Crear un prompt activo

El enunciado no define la ruta exacta para PromptsConfig (queda a criterio del equipo). Ajusten el path de abajo al que ustedes hayan definido — lo más común es

POST {{base_url}}/prompts.

```
{
  "tipo_lectura": "Personalidad",
  "estructura_prompt": "Actúa como un numerólogo experto. Con el camino de vida {{numero_camino_vida}}, el número de expresión {{numero_expresion}} y el deseo del alma {{numero_alma}}, genera una lectura de personalidad cálida y profesional en español, de máximo 200 palabras.",
  "activo": true
}
```

- Esperado: **201 Created**
- Guarden el `_id` en `prompt_id` con el mismo patrón de Tests visto antes
- Prueba útil: creen un **segundo** prompt con `"activo": false` y verifiquen que el endpoint de generación (sección 6.2) nunca lo selecciona — así confirman que el filtro `{ activo: true }` en la consulta realmente funciona

6.2 Generar una lectura con Gemini — POST

{{base_url}}/lecturas/generar/{{usuario_id}}

- No lleva body: el endpoint es orquestador — internamente debe: 1) buscar la matriz del usuario, 2) cargar el prompt activo, 3) armar el prompt final reemplazando los números, 4) llamar a la API de Gemini, 5) guardar el resultado en `LecturasIA`
- Esperado: **201 Created** con `contenido_generado` (el texto que devolvió Gemini), `tokens_consumidos`, y las referencias `usuario_id` y `prompt_usado_id`
- **Esta petición tarda más** que las demás (llamada externa a Gemini) — no la cancelen antes de 5-10 segundos

Errores esperables en este endpoint específicamente:

Código	Causa probable
401 / 403 desde Gemini (visible en consola del servidor)	GEMINI_API_KEY mal copiada o vencida en el <code>.env</code>
404 con mensaje de "matriz no encontrada"	Intentaron generar la lectura sin haber ejecutado antes el POST <code>/numerologia/calcular/:usuario_id</code>
404 con mensaje de "prompt no encontrado"	No existe ningún <code>PromptsConfig</code> con <code>activo: true</code> en la base de datos
500 genérico	Revisen la consola del servidor — puede ser un error de formato en la respuesta de Gemini que el código no está parseando bien

6.3 Consultar historial — GET

{{base_url}}/lecturas/historial/{{usuario_id}}

- Esperado: **200 OK** con un **array** de lecturas (repitan el paso 6.2 dos o tres veces para tener

varias)

- Si el controller usa `.populate("prompt_usado_id")`, cada lectura del historial debe traer embebido el `tipo_lectura` del prompt que se usó — así en el historial se puede saber si fue una lectura de "Amor", "Anual" o "Personalidad" sin tener que consultar `PromptsConfig` aparte

7. BitacoraTransacciones — cómo probarla

Esta colección normalmente **no se llena manualmente desde Postman**: la idea del enunciado es que sea un **log de auditoría automático**, es decir, un middleware que registre cada petición (endpoint, estado_http, tiempo_ejecucion_ms, fecha) sin que el aprendiz tenga que llamarlo a propósito.

Cómo verificarla en Postman:

1. Ejecuten cualquiera de los endpoints anteriores normalmente (ej. el GET `/usuarios/:id`)
2. Si construyeron un endpoint de consulta para revisar la bitácora (ej. GET `{{base_url}}/bitacora`), llámenlo después y confirmen que apareció un registro nuevo correspondiente a la petición del paso 1
3. Si no construyeron ese endpoint de consulta, verifiquen directamente en MongoDB Compass o mongosh que la colección `bitacoratransacciones` está creciendo con cada petición que hacen desde Postman

8. Tabla resumen del flujo completo a probar

#	Método	Endpoint	Depende de
1	POST	<code>/usuarios</code>	—
2	GET	<code>/usuarios/:id</code>	usuario creado en 1
3	POST	<code>/numerologia/calcular/:usuario_id</code>	usuario creado en 1
4	GET	<code>/numerologia/matriz/:usuario_id</code>	matriz creada en 3
5	POST	<code>/prompts</code> (ruta propia del equipo)	—
6	POST	<code>/lecturas/generar/:usuario_id</code>	matriz (paso 3) + prompt activo (paso 5)
7	GET	<code>/lecturas/historial/:usuario_id</code>	al menos una lectura generada en 6

Este orden **no es opcional**: si prueban el paso 6 antes que el 3 y el 5, va a fallar por diseño (el endpoint depende de que existan esos datos previos) — y esa es justo la prueba de que la lógica de orquestación está bien construida.

9. Checklist final

- ☐ `Collection Numerologia` API con las 3 carpetas del enunciado
- ☐ `Environment Local` con `base_url`, `usuario_id`, `prompt_id`
- ☐ Usuario creado y probado el caso de email duplicado (validación de unicidad)
- ☐ Cálculo de matriz probado, y los números verificados a mano contra la respuesta de la API
- ☐ Consulta de matriz probada con y sin `populate` para notar la diferencia
- ☐ Al menos un `PromptsConfig` con `activo: true` creado antes de probar la generación de lecturas
- ☐ Generación de lectura probada, incluyendo los 3 casos de error de la tabla de la sección 6.2 (sin matriz, sin prompt activo, API key inválida)
- ☐ Historial probado con más de una lectura generada, confirmando que `populate` trae el `tipo_lectura` de cada una
- ☐ Verificaron que `BitacoraTransacciones` efectivamente se llena en cada petición (por endpoint propio o revisando la base de datos directamente)
- ☐ Revisaron la consola de la terminal (no solo Postman) en cada error 500 para leer el stack trace real